

analysis

图 1 显示, `lighttpd` 会将 `HNAP1` 请求转发给 `/bin/prog.cgi`。因此, 针对 `/HNAP1/dlcfg.cgi`、`/HNAP1/dllog.cgi` 和 `/HNAP1/dlquickvpnsettings.cgi` 的未授权请求, 最终都会到达这个存在漏洞的程序 (二进制文件)。

```
fastcgi.debug = 1
fastcgi.server = (
    (
        "socket" => "/var/prog.fcgi.socket-0",
        "check-local" => "enable",
        "bin-path" => "/bin/prog.cgi",
        "idle-timeout" => 10,
        "min-procs" => 1,
        "max-procs" => 1
    ),

```

图 2 显示, `websSecurityHandler` 调用了位于 `0x423e04` 的身份验证分发函数。该函数负责决定是拒绝、验证传入的 `HNAP1` 请求, 还是通过特殊规则直接放行。

```
int __fastcall websSecurityHandler(int a1, int a2)
{
    int v7; // $v0
    int v9; // [sp+18h] [-30h]
    _BYTE v10[24]; // [sp+30h] [-18h] BYREF

    v9 = 0;
    memset(v10, 0, 20);
    strncpy(v10, "127.0.0.1", 20);
    if ( *(_DWORD *)(a1 + 228) )
    {
        v7 = strlen(*(_DWORD *)(a1 + 228));
        if ( !strcmp(*(_DWORD *)(a1 + 228), v10) )
            return 0;
    }
    if ( sub_423E04(a1) )
        return 1;
    return v9;
}
```

图 3 显示, 位于 `0x423e04` 的代码调用了位于 `0x423ca8` 的白名单检查器。当检查器返回 1 时, 身份验证逻辑会允许请求继续执行, 而不是将其拒绝。

```
else if ( sub_422FEC(a1) )
{
    if ( sub_423CA8(a1) == 1 )
    {
        websDefaultHandler(a1, 0, 0, 0, "/Index.html", "/Index.html", *(_DWORD *)(a1 + 244));
        return 1;
    }
    else
    {
        return sub_4232E4(a1);
    }
}
```

图 4 显示, 白名单检查器使用 `sscanf` 函数及模式 `"/HNAP1/%s"` 来解析攻击者控制的 `HNAP1` 路径。因此, 用于白名单比较的 CGI 名称实际上是由 HTTP 请求的 URL 控制的。

```

1 int __fastcall sub_423CA8(int a1)
2 {
3     unsigned int i; // [sp+18h] [-80Ch]
4     _BYTE v3[2056]; // [sp+1Ch] [-808h] BYREF
5
6     memset(v3, 0, 2048);
7     if ( *(_DWORD *)(a1 + 224) )
8     {
9         for ( i = 0; i < 3; ++i )
10        {
11            sscanf(*(_DWORD *)(a1 + 224), "/HNAP1/%s", v3);
12            if ( !strcmp(v3, &downloadurls[64 * i]) )
13                return 1;
14        }
15    }
16    return 0;
17 }

```

图 5 显示，程序使用 `strcmp` 函数将解析出的 CGI 名称与 `downloadurls` 表中的条目进行比较。如果该名称与白名单中的 CGI 匹配，函数将返回 1。

```

1 int __fastcall sub_423CA8(int a1)
2 {
3     unsigned int i; // [sp+18h] [-80Ch]
4     _BYTE v3[2056]; // [sp+1Ch] [-808h] BYREF
5
6     memset(v3, 0, 2048);
7     if ( *(_DWORD *)(a1 + 224) )
8     {
9         for ( i = 0; i < 3; ++i )
10        {
11            sscanf(*(_DWORD *)(a1 + 224), "/HNAP1/%s", v3);
12            if ( !strcmp(v3, &downloadurls[64 * i]) )
13                return 1;
14        }
15    }
16    return 0;
17 }

```

图 6 展示了 `downloadurls` 白名单表。该表中包含了 `dlcfg.cgi`、`dllog.cgi` 和 `dlquic` `kvpnsettings.cgi`，这些请求随后会被分发给敏感的下載处理程序。

```

.data:004D1FCF .byte 0
.data:004D1FD0 .byte 0x64 # d
.data:004D1FD1 .byte 0x6C # l
.data:004D1FD2 .byte 0x6C # l
.data:004D1FD3 .byte 0x6F # o
.data:004D1FD4 .byte 0x67 # g
.data:004D1FD5 .byte 0x2E # .
.data:004D1FD6 .byte 0x63 # c
.data:004D1FD7 .byte 0x67 # g
.data:004D1FD8 .byte 0x69 # i

```

如图 7 所示，`downloadFile` 函数会将请求的 CGI 名称与 `downloadurls` 表进行匹配，然后从位于 `0x4d2050` 的处理器表 (handler table) 中调用相应的函数指针。这就将白名单中的 URL 与那些敏感的下載处理程序关联了起来。

0x4d2050 -> 0x4297d0

0x4d2054 -> 0x42a328

0x4d2058 -> 0x42a500

```

BOOL __fastcall downloadFile(int a1)
{
    BOOL result; // $v0
    unsigned int i; // [sp+18h] [-Ch]

    for ( i = 0; ; ++i )
    {
        result = i < 3;
        if ( i >= 3 )
            break;
        if ( !strcmp(a1, &downloadurls[64 * i]) )
        {
            off_4D2050[i]();
            return 0;
        }
    }
    return result;
}

```

如图 8-1 所示，dlcfg.cgi 处理程序会准备 config.bin 文件并将其作为可下载文件返回。该文件可能包含设备配置、无线凭证（如 Wi-Fi 密码）以及其他敏感设置。

```

v0 = strlen(v0 + 2);
v12 = crc32(0, v6, v9);
v3 = FCGI_fopen("/etc_ro/lighttpd/www/web/config.bin", "wb");
if ( v3 )
{
    FCGI_fwrite(v13, 4, 1, v3);
    FCGI_fwrite(&v12, 4, 1, v3);
    v0 = strlen(v6 + 2);
    FCGI_fwrite(v6, v0 + 8, 1, v3);
    FCGI_puts(
        "Pragma: no-cache\r\n"
        "Cache-Control: no-cache\r\n"
        "Content-Type: application/force-download\r\n"
        "Content-Transfer-Encoding: binary\r\n"
        "Content-Disposition: attachment; filename=\"config.bin\"\r\n");
    FCGI_fwrite(v13, 4, 1, (char *)&fcgi_sF + 8);
    FCGI_fwrite(&v12, 4, 1, (char *)&fcgi_sF + 8);
}

```

如图 8-2 所示，dllog.cgi 处理程序会复制 /var/log/messages 文件，并将内存、进程以及网络接口信息追加到导出的日志文件中。因此，未经身份验证的调用者（攻击者）可以借此获取设备敏感的运行时信息。

```

int sub_42A328()
{
    system("cp /var/log/messages /etc_ro/lighttpd/www/web/messages");
    system("free >> /etc_ro/lighttpd/www/web/messages");
    system("ps >> /etc_ro/lighttpd/www/web/messages");
    system("ifconfig >> /etc_ro/lighttpd/www/web/messages");
    sleep(1);
    FCGI_printf("HTTP/1.1 %d %s\n", 200, "OK");
    FCGI_printf("Pragma: no-cache\n");
    FCGI_printf("Cache-Control: no-cache\n");
    FCGI_printf("Content-Type: application/force-download\n");
    FCGI_printf("Content-Transfer-Encoding: binary\n");
    FCGI_printf("Content-Disposition: attachment; filename=\"messages\"\n");
    FCGI_printf("\n");
    return sub_429FF0("/etc_ro/lighttpd/www/web/messages", (char *)&fcgi_sF + 8);
}

```

如图 8-3 所示，dlquickvpnsettings.cgi 处理程序会读取 /tmp/vpnprofile.mobileconfig 文件，并将其内容直接写入 HTTP 响应中。如果该文件存在，则可能会暴露 VPN 配置文件数据。

```

if ( v2 )
{
    if ( FCGI_fread(v2, 1, v4, v3) == v4 )
    {
        *(_BYTE *)(v2 + v4) = 0;
        FCGI_printf("HTTP/1.1 %d %s\n", 200, "OK");
        FCGI_printf("Pragma: no-cache\n");
        FCGI_printf("Cache-Control: no-cache\n");
        FCGI_printf("Content-Type: application/force-download\n");
        FCGI_printf("Content-Transfer-Encoding: binary\n");
        FCGI_printf("Content-Disposition: attachment; filename=\"vpnprofile.mobileconfig\"\n");
        FCGI_printf("\n");
        v0 = strlen(v2);
        FCGI_fwrite(v2, 1, v0, (char *)&fcgi_sF + 8);
    }
}

```

